altairsim — Changelog

2026-07-23 · 76250ff

Changelog

Changelog

Every release of **altairsim**, newest first. Each entry is the short version — what you can do here that you could not do in the release before it. The User Manual describes the program as it is now; this document is the record of how it got there.

Unreleased

Reach the host shell without leaving the machine — `!`

A monitor line that begins with `!` runs the rest of it in **your host shell** and returns you to the prompt when it finishes — `!ls`, `!cp game.dsk save.dsk`, `!vi HELLO.PRN` to edit a file in place. Everything after the `!` is passed through verbatim, spaces and all, and an interactive program like `vi` gets a normal terminal because the monitor is not holding the keyboard when it hands off. It runs with *your* privileges, not the guest's, and the machine keeps its state underneath — `!` borrows you, not the processor. A bare `!` just shows the form.

Save the machine and pick it back up — `SNAPSHOT` and `RESTORE`

`SNAPSHOT <file>` writes the whole machine's **state** to a small, portable, CRC-checked file: the CPU — down to the hidden micro-state a register dump never shows, the EI and interrupt-acknowledge latches, the Z80's `WZ`, `IFF2` and interrupt mode — the clock, and every board's registers, latches and RAM. `RESTORE <file>` reads it back into a machine of the same shape. A snapshot is *state*, not configuration, so `RESTORE` validates the file's checksum, version and board topology **before** it applies a single byte: a corrupt or mismatched file is refused with the reason, and your running machine is left untouched. This is the largest piece of the design's replay groundwork; the deterministic `RECORD` / `REPLAY` half stays reserved and is unblocked by it.

The disassembler reads symbols

Load an assembler listing and `DISASM` stops speaking in hex. A program label heads its own line the way a listing prints it, and a 16-bit operand shows the name it points at — `CALL 0005` becomes `CALL BDOS`, `JMP 0100` becomes `JMP LOOP`. Single-stepping shows it too, on the `STEP` and `REGS` line. A leading `LABEL:` line comes from real program labels only — an `EQU` never heads a line, because a constant that merely equals a code address must not masquerade as one — but an operand *reference* uses any symbol, so `CALL BDOS` works even though `BDOS` is an `EQU`, and a real label wins when the two share a value. Only a 16-bit operand is an address: a byte like `IN 10` stays a number. Nothing changes until you `SYMBOLS LOAD`.

`examples/debugger/` is new alongside it — a 46-byte program with its listing and Intel HEX, a machine that comes up at the monitor prompt, and a walkthrough (shipped as `README.pdf` as well as Markdown) from `SYMBOLS LOAD` through symbolic `DISASM`, single-stepping, breaking on a label *by name*, and running it until it prints `HELLO, WORLD`. It is the fifth example in the package.

The examples boot themselves — a new `TYPE` command

`TYPE` injects keystrokes into the console the way a key from the terminal or the VDM window does — type-ahead the guest reads when it next looks, with `\r`, `\n`, `\t`, `\\` and `\"` decoded. A machine file's `startup` runs *monitor* commands and so could never reach a program running inside the guest; a `TYPE` before the `RUN` that boots it leaves the command waiting at the first prompt. That is what now lets the four Sol-20 cassette games — `trek80`, `atc`, `pacman`, `raiders` — mount their tape, type their own `XE <name>` and come up on their own at the Sol's real 2.045 MHz, instead of dropping you at the SOLOS prompt to launch them by hand.

The bus says when a board isn't there — `SET BUS UNCLAIMED`

A guest that reached for a board that isn't in the backplane used to read `0xFF` forever and hang with nothing on the screen. `SET BUS UNCLAIMED WARN` now logs the reach — `warning: OUT 0FE <- 01 at PC=0113: no board decodes port 0xFE` — and runs on; `HALT` logs it and stops the guest at that instruction, so `SHOW REG` and `DUMP` see the machine exactly as it wedged. It is I/O only (a guest scans memory constantly), reported once per port and direction per `RUN`, and `Silent` by default so nothing that was quiet becomes noisy.

One command from clone to binary

`build.sh`, and its plain-PowerShell twin `build.bat`, takes a fresh clone to a built binary in one command — no flags to choose, no generator to pick — and if CMake is missing it prints how to get it for your platform rather than failing deep in a configure. SDL3 stays optional: a plain build needs nothing installed, and `--with-sdl` links a private static SDL3 so the binary carries its own. For a release, `tools/build-checksums.sh` writes `dist/SHA256SUMS`, refusing unless all four platform archives of a version are present — a partial checksum file is worse than none.

0.3.0

0.3.0 adds no machines and no boards. It changes one thing, and it is the thing the manual has described all along: the copy you download now opens the video window.

The window the manual documents is finally in the package

Every release through 0.2.0 shipped a **headless** binary. SDL3 was not compiled into it, so the VDM-1 and Sol-20 windows the boards and configuring chapters describe at length could not open from anything you were handed — the machines ran, and drew nothing. `SHOW VERSION` said so in a row nobody had reason to read:

```
altairsim> SHOW VERSION
  altairsim  0.3.0
  video      SDL3 -- windowed        <- 0.2.0 and before, the download read: none -- headless
  commit     v0.3.0
  tree       clean
```

0.3.0 is the first release whose downloaded package reads `SDL3 -- windowed`. Run `altairsim sol20`, and a window opens. That is the headline, and most of the rest of this entry is how it was made true on every platform at once.

Nothing to install, on any of the four

The packages are built on the hardware they target now, rather than assembled by CI, and SDL3 is **linked statically into the binary**. So there is no `SDL3.dll` to sit beside the `.exe`, no `.framework`, no `libSDL3.so.0`, and nothing to install before the program runs — the claim altairsim has always made for itself is now true of its video too. On Windows the C runtime is static as well, so a clean machine that has never had a compiler on it runs the `.exe` with no Microsoft redistributable to chase.

The one visible cost is a single extra file in the archive — `LICENSE-SDL3`, SDL3's zlib licence, because SDL3's code now travels inside the binary and its licence travels with it.

macOS ships as two builds, each tested on its own hardware

0.2.0 shipped one *universal* macOS archive. 0.3.0 ships two — `altairsim-0.3.0-macos-arm64` and `altairsim-0.3.0-macos-x86_64` — and the reason is honesty, not size. The universal binary's Intel half had been built and tested by nobody, because the machine that produced it was Apple Silicon; the previous two releases said so in their own notes. Each of the two archives is now built **and** run on the architecture it targets, so the Intel download is exercised on an Intel Mac before it ships. Take the one that matches your Mac; `altairsim --version` and the download filename both name the architecture.

Windows, proven end to end

The Windows package is built with MSVC, links SDL3 and the C runtime statically, passes the full test suite on the machine that builds it, and opens a real VDM-1 window that a person has

sat in front of. `dumpbin /dependents` on the shipped `.exe` shows system DLLs only — no `SDL3.dll`, no `VCRUNTIME140`. It is held to the same bar as the other three, and it is no longer an asterisk on the release.

The four downloads

Platform	File
macOS Apple Silicon	`altairsim-0.3.0-macos-arm64.tar.gz`
macOS Intel	`altairsim-0.3.0-macos-x86_64.tar.gz`
Linux x86_64	`altairsim-0.3.0-linux-x86_64.tar.gz`
Windows x86_64	`altairsim-0.3.0-windows-x86_64.zip`

Each holds the program, this changelog, the **User Manual**, `USING-ALTAIRSIM.md`, both licences, and `examples/` — four machines that boot, media included. Unzip it and run it; nothing needs fetching first.

What did not change

The simulator itself is byte-for-byte the machine 0.2.0 was — the same sixteen machines, the same boards, the same two CPU cores. The holes named in the manual's introduction are still holes: no snapshot, no replay, the six reserved monitor verbs still reserved, still no audio. Everything that moved is in the box the program arrives in.

0.2.0

The second release. It added machines and made the video window behave like a window — but note that the packages were still **headless** (see 0.3.0): everything below was true of a build made *with* SDL3, which is not what the archives carried until 0.3.0.

Three more monitors that boot from a bare command line

`amon`, `acuter` and `cdbl` became machines, so Martin Eberhard's Altair ROMs boot by name — nothing to fetch, nothing to mount:

```
altairsim amon          AMON 3.1 in a 4K EPROM at F000 -- a full-featured Altair monitor
altairsim acuter        ACUTER at F000 -- CUTER on a plain Altair, driving a terminal
altairsim cdbl          the default machine, with the Combo Disk Boot Loader in the socket
```

The ROM images shipped in 0.1.0 already; what was new is that each got a machine built around it — the whole distance between shipping an image and being able to run it. `hdbl` was deliberately left out: it boots an 88-HDSK hard disk, and there is no 88-HDSK board here. **Sixteen machines** became built in.

The video window behaves like a window

- **It does not steal the keyboard when it opens.** The terminal keeps the keys while you type at the monitor prompt; `SET DISPLAY focus=on` hands them to the guest, stopping it hands them back.
- **It is named after the machine**, so `sol20` and `vdm1` are two windows you can tell apart.
- **It is sized to fit the screen it opened on**, and **arrows and HOME reach the guest**.
- **Closing it stops the guest**, instead of leaving a machine running with nothing to draw on.

Two of those were bugs worth naming: typed input could lag a whole frame, and the VDM-1 could repaint hundreds of times per emulated millisecond. Both gone. The VDM-1's cursor now blinks on the board's own oscillator — wall-clock time, as the hardware did.

Disk BASIC, and smaller things

- `examples/diskbasic` boots **Altair Disk BASIC 4.1** off a floppy, media included — a fourth worked example alongside CP/M, cassette BASIC and the Sol-20.
- A binary now names the commit it was built from: `SHOW VERSION` and `--version` carry it, so a report against a nightly or a CI artifact can be traced to the code that produced it.
- `writeprotect` is accepted wherever `readonly` is, in machine files and at `SET`.
- The manual and the program say **board**, not card.

0.1.0

The first release — a simulator for the MITS Altair 8800 and the S-100 bus, in C++20, with **nothing to fetch**: the TOML parser, the JSON encoder and the line editor are all in-tree, so a fresh clone builds with a C++20 compiler and CMake and no network.

Two validated CPU cores

Both cleared their gate before a single board was built on them, and for the release the exercisers were re-run on all three platforms — roughly 15 billion instructions each:

- **8080** — TST8080, 8080PRE, CPUTEST, 8080EXM

- **Z80** — ZEXDOC, ZEXALL

Fourteen boards, thirteen machines

88-2SIO · 88-SIO · 88-ACR · 88-DCDD · 88-MDS · 88-VI/RTC · 88-C700 · front panel · memory · 8080 CPU · Z80 CPU · VDM-1 · Processor Technology Sol-PC · Host Bridge (our own design).

CP/M 2.2 cold-boots from both 8-inch and 5.25-inch disks. MITS 4K and 8K BASIC and Programming System II load from cassette — including **real `.WAV` audio, played and recorded**, at measured CUTS/ACR parameters.

Debugging, and an assistant that can drive it

Breakpoints, watchpoints, tracepoints, conditional breaks (`BREAK … IF`), execution history, and symbolic reference loaded from `.PRN` and `.SYM` files — DDT and SID run under it, self-modifying RST 7 breakpoints and all. An **MCP server** lets an AI assistant drive a running guest.